

Vanier College

201-SND-VA ARTIFICIAL INTELLIGENCE

Sect. 00001

LIA Project

Understanding Grokking in AI

Presented to I.T Ivanov

Frank Michael Castillo

May 14th, 2026

Introduction

Grokking is a phenomenon in machine learning where a neural network suddenly begins generalizing long after it appears to have overfit the training data. In many cases, the model first memorizes the training examples and achieves extremely low training loss while the test loss remains high. However, after prolonged optimization, the model unexpectedly transitions to a cleaner and more generalizable internal solution. This project investigates grokking in small transformer models trained on modular arithmetic tasks. Using mechanistic interpretability techniques, the project studies how the internal representations of the network evolve during training and how Fourier structure emerges inside the model. In this project, coding notebooks provided by the teacher were used to train and analyze the transformer models. These notebooks served as the starting point for the experiments and were executed using Antigravity software.

- Training model: https://mai.trigraph.org/Jupyter/Grokking_Training.ipynb
- Analyzing model: https://mai.trigraph.org/Jupyter/Grokking_Analysis.ipynb

Objective of the Project

The objective of this project is to reproduce and analyze the grokking phenomenon in small transformer models using mechanistic interpretability techniques.

1. Reproducing Grokking in Modular Arithmetic

The first objective of this project was to reproduce the grokking phenomenon in a small transformer trained on a modular arithmetic task. The goal was to observe whether the model would first memorize the training data before later transitioning to a more generalizable algorithmic solution. To investigate this behavior, a one-layer transformer was trained on modular addition using the prime modulus:

- $p = 113$

The model was trained for:

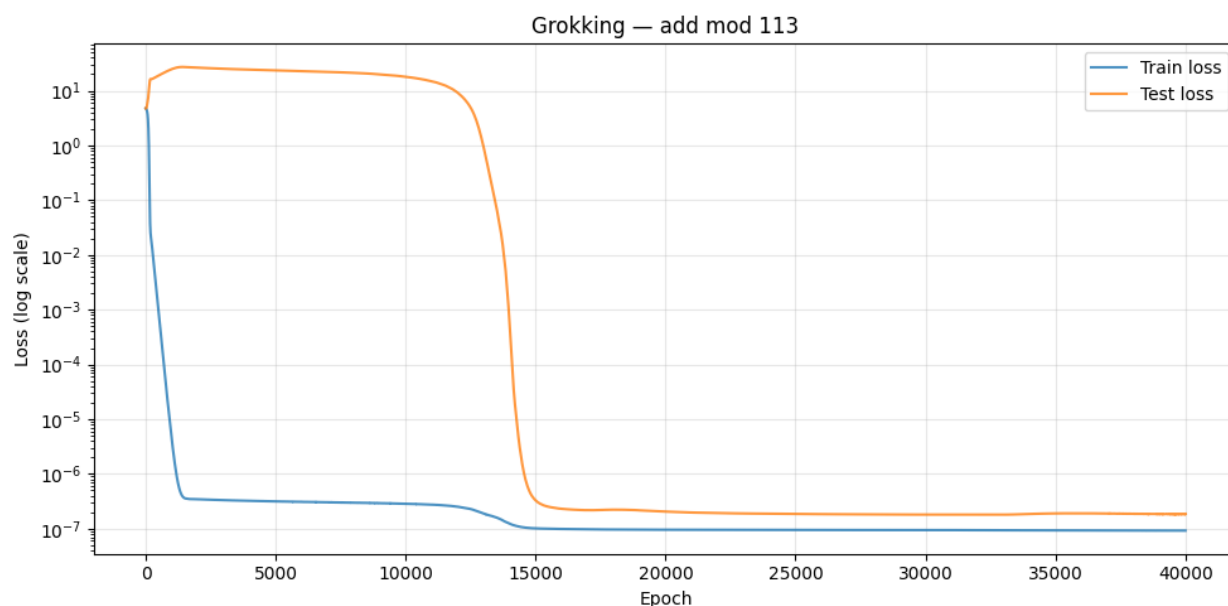
- 40,000 epochs

With a training fraction of:

- 0.3

meaning that the transformer only saw 30% of all possible modular arithmetic examples during training.

The resulting loss curves reproduced the characteristic grokking phenomenon. Early in training, the training loss rapidly approached zero while the test loss remained high, indicating memorization without generalization. Around epochs 12,000–15,000, the test loss suddenly collapsed, and the model began generalizing successfully.



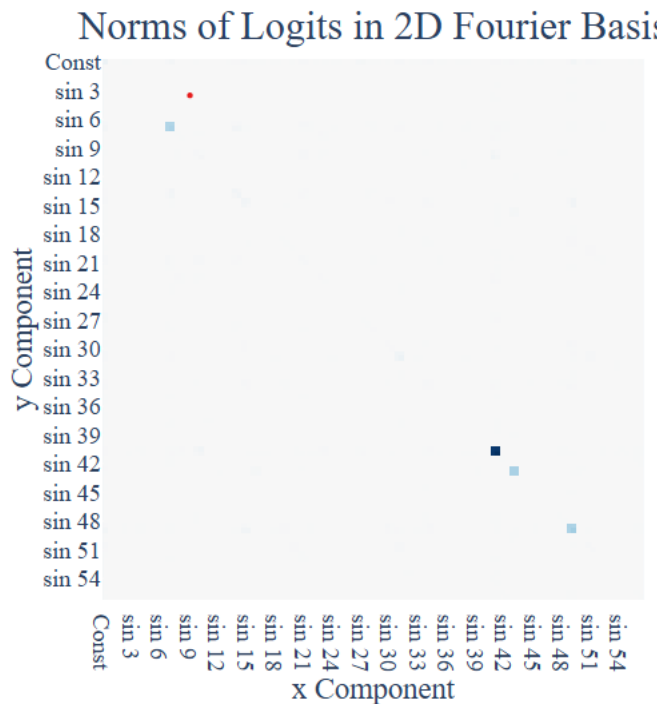
Graph: Training and test loss curves

These results demonstrate delayed generalization: the transformer first memorized the training data before later discovering a cleaner and more structured representation of modular arithmetic.

2. Fourier analysis of Embeddings and MLP activations

The second objective of this project was to analyze the internal representations learned by the transformer after the grokking transition. Using mechanistic interpretability techniques, the embeddings, attention heads, and MLP neuron activations of the model were examined in order to understand how the transformer internally represents modular arithmetic. The learned embedding matrix was extracted after training. The embedding tensor had dimensions: [113,512]

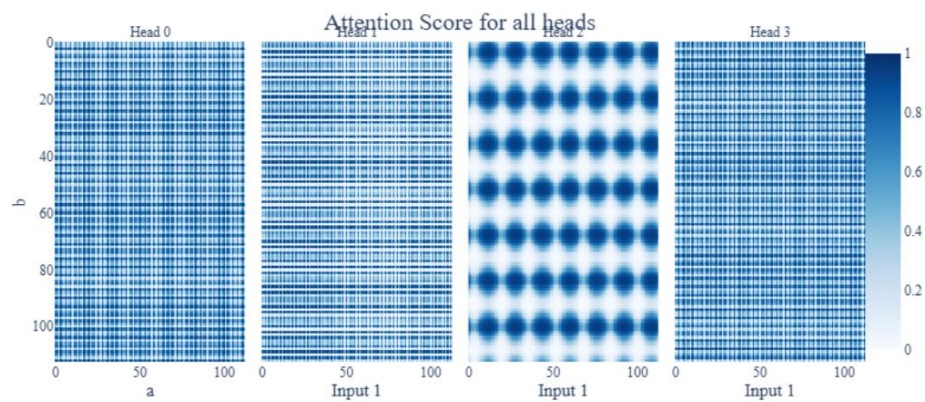
A 1D Discrete Fourier Transform was applied to the embeddings to study their frequency structure. The Fourier-basis heatmaps showed that only a small number of frequencies had large magnitude while most frequencies remained near zero.



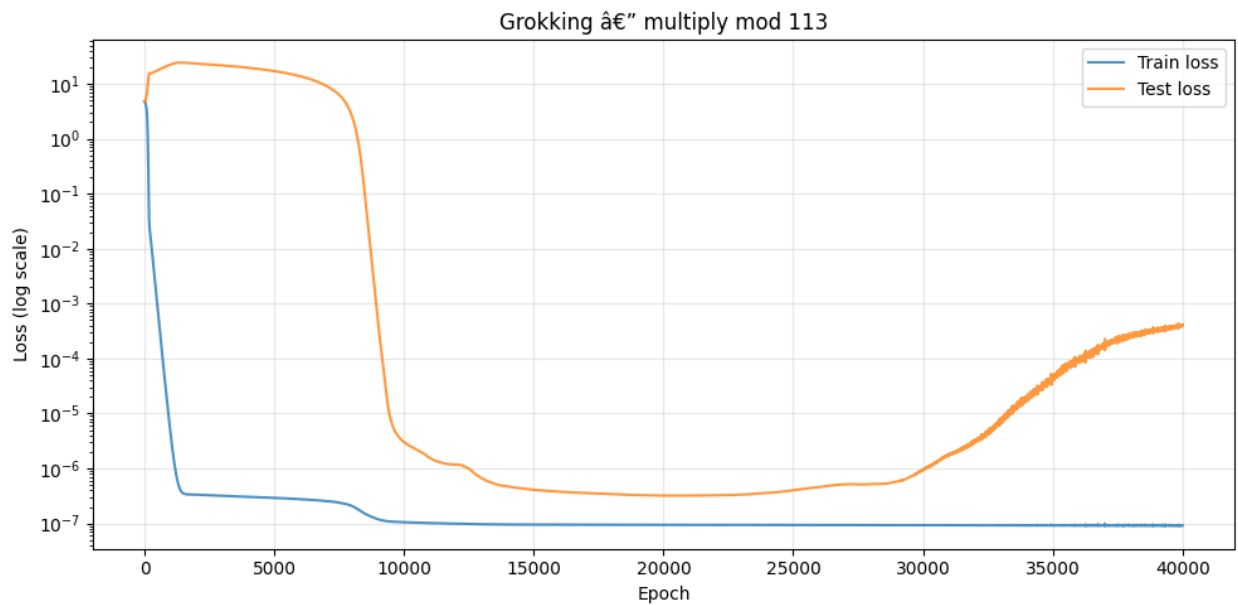
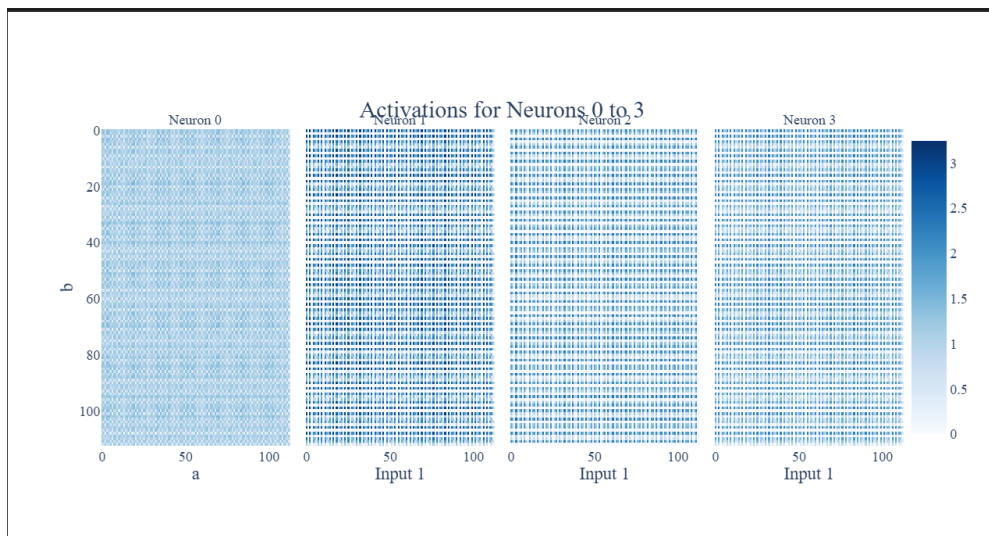
Graph: Norms of logits in 2D Fourier Basis

The Fourier-basis heatmap is mostly close to zero, with only a few isolated frequency components having large magnitude. This sparse structure demonstrates that the transformer did not memorize examples independently. Instead, it learned compact periodic representations aligned with the mathematical structure of modular arithmetic.

Attention-head heatmaps also displayed highly organized stripe-like patterns across the modular input space.



Heatmap: Attention Score for all heads



corresponding to all possible ordered input pairs (a, b) .

Heatmap: Activations for Neurons 0 to 3

The neuron activation heatmaps showed strong periodic structures rather than random activation behavior.

The neuron activation heatmaps showed strong periodic structures rather than random activation behavior. Several neurons behaved like detectors for specific Fourier frequencies.

Together, these results demonstrate that the transformer learned a sparse set of key periodic frequencies instead of relying purely on memorization.

3. The three phases of Grokking

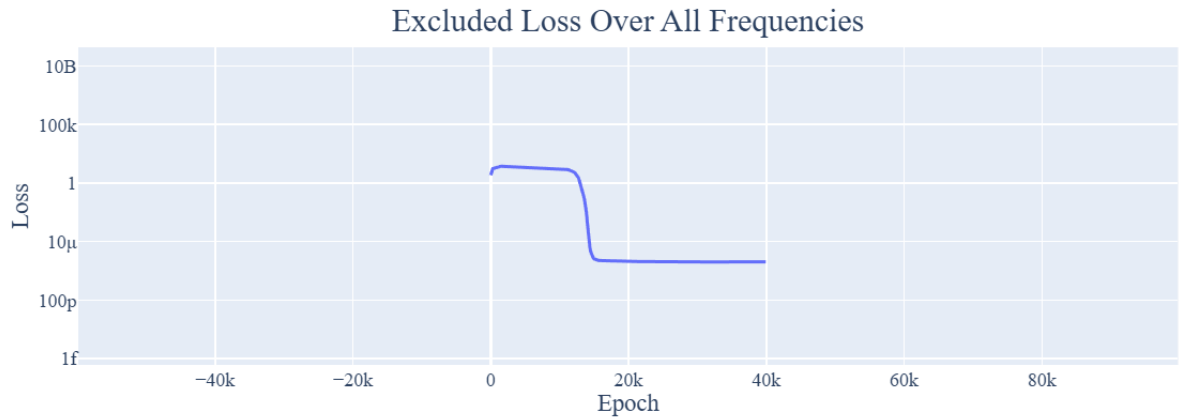
The third objective of this project was to study how the internal behavior of the transformer evolved throughout training. Several interpretability metrics were tracked in order to better understand the transition from memorization to generalization. The following metrics were analyzed during training:

- Excluded loss over all frequencies
- Pure restricted loss
- Total sum of squared weights
- Test loss

These metrics revealed three distinct phases of grokking.

1. Memorization phase

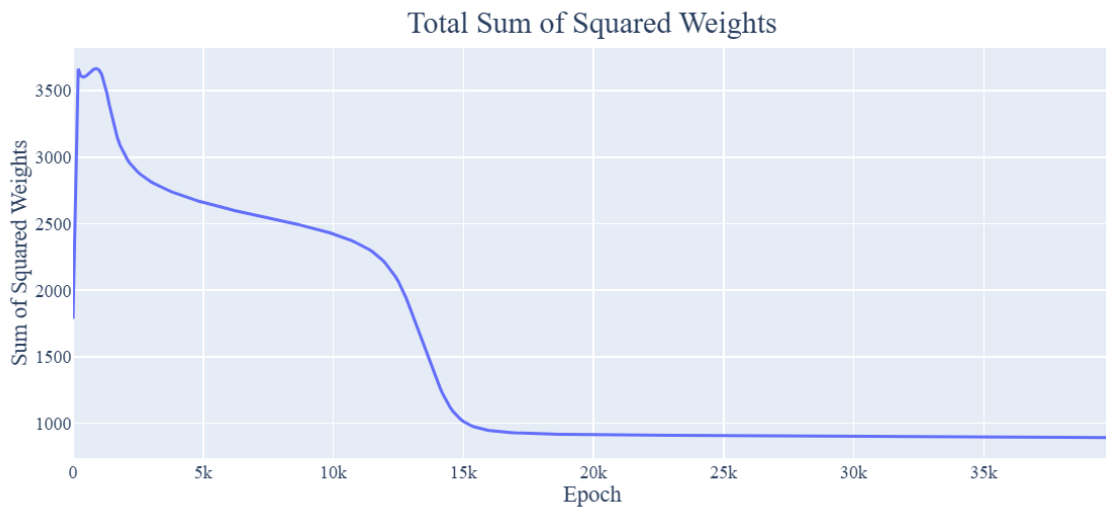
During the early stages of training, the excluded loss decreased while the test loss remained high. This indicates that the model primarily memorized the training examples without learning a generalizable algorithm.



Graph: Excluded loss over all frequencies.

2. Circuit formation phase

As training continued, the restricted loss and the total sum of squared weights began decreasing. This suggests that the model was gradually forming a cleaner and more compact Fourier-based algorithmic circuit. Graph: Pure restricted



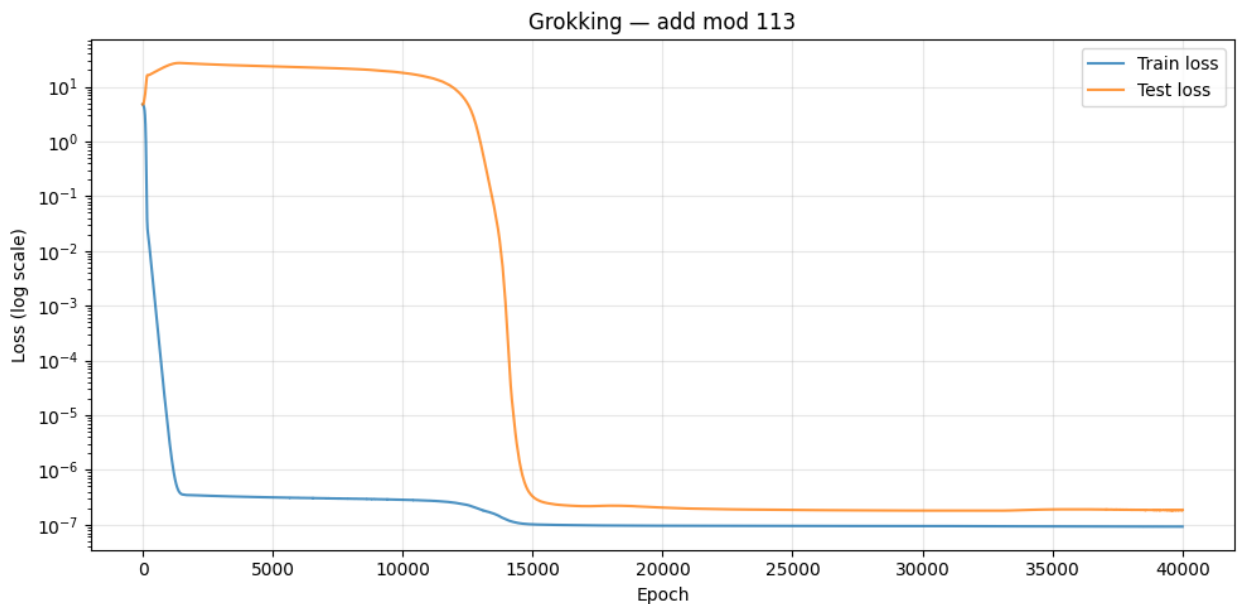
Graph: Total sum of squared weights



Graph: Pure restricted Loss

3. Cleanup Phase Around

Around epochs 12,000–15,000, the test loss suddenly collapsed and the model began generalizing successfully. epochs 12,000–15,000, the test loss suddenly collapsed and the model began generalizing.



Graph: Training and test loss curves

The results support the interpretation that grokking is a gradual internal reorganization from memorization toward a structured and generalizable algorithmic representation

4. Effects of training data fraction on generalization

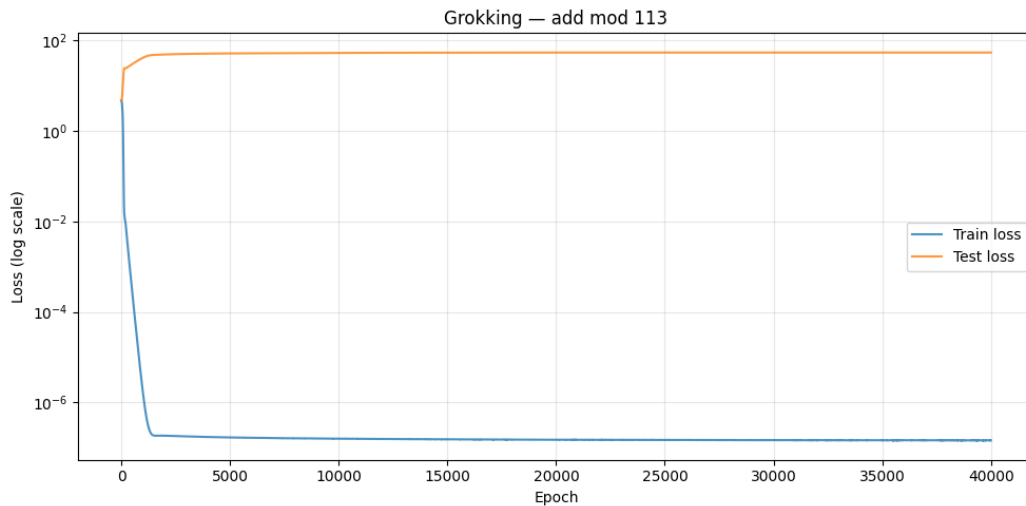
The fourth objective of this project was to investigate how the amount of training data affects the grokking phenomenon. To study the effect of data scarcity on generalization, the transformer was trained multiple times using different training fractions while keeping:

- ($p = 113$)
- 40,000 epochs

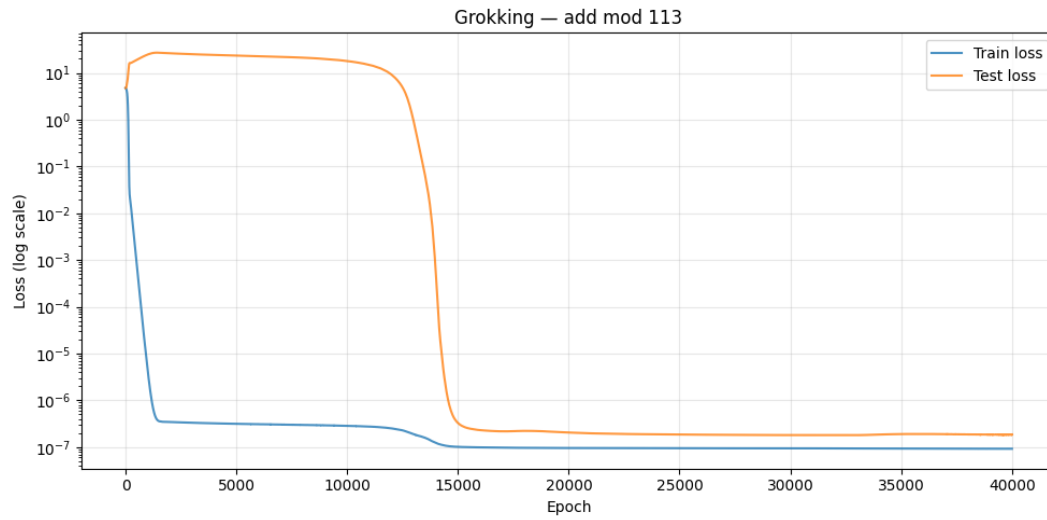
Constant across all experiments

The following results were obtained:

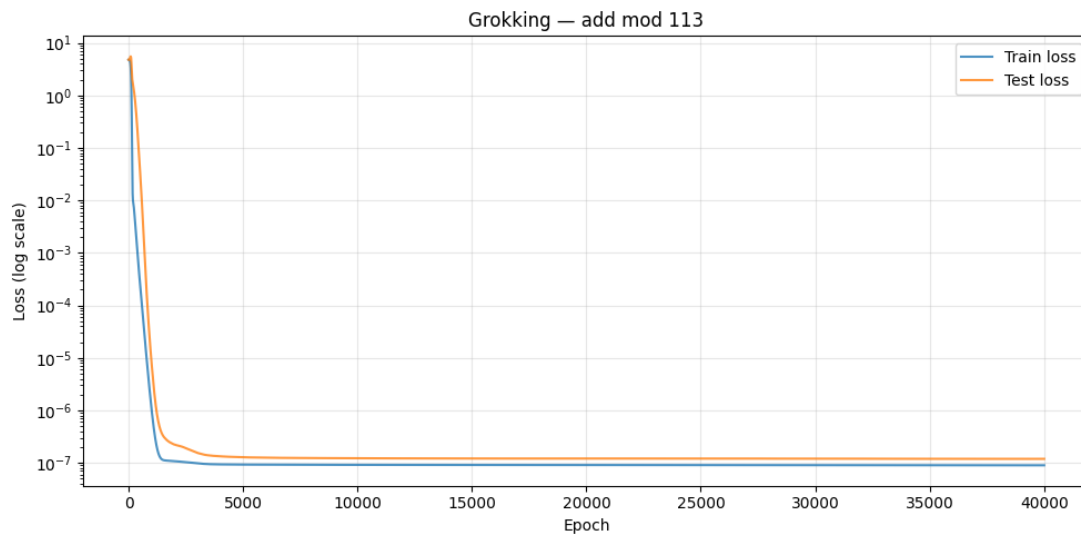
Training Fraction	Generalization Behavior
0.1	Never grokked
0.3	Grokking around 12,000–15,000 epochs
0.6	Generalized almost immediately
0.9	Generalized immediately



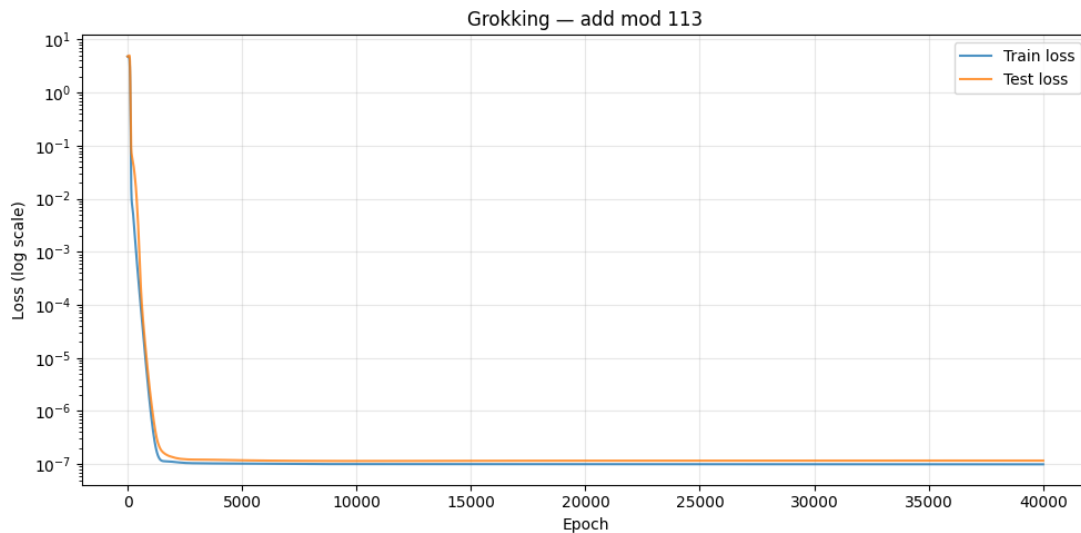
Graph: Training and test loss curves (training fraction = 0.1)



Graph: Training and test loss curves (training fraction = 0.3)



Graph: Training and test loss curves (training fraction = 0.6)



Graph: Training and test loss curves (training fraction = 0.9)

The experiments show that delayed grokking occurs primarily in an intermediate regime of data scarcity. When the training fraction is very small, the model struggles to discover a stable generalizable circuit. As the training fraction becomes larger, the model generalizes much earlier and the delayed “aha moment” gradually disappears.

4. AI alignment and Mechanistic interpretability

The fifth objective of this project was to explore the relationship between AI alignment and mechanistic interpretability, two important areas in modern AI safety research. AI alignment is the problem of ensuring that artificial intelligence systems behave safely and pursue intended goals. As neural networks become increasingly powerful, understanding how they internally reason becomes increasingly important. A model may produce correct outputs while internally relying on strategies that are difficult to detect or understand.

Mechanistic interpretability attempts to reverse engineer neural networks by studying embeddings, activations, attention heads, and learned circuits. Rather than treating neural networks as black boxes, interpretability aims to understand the actual algorithms learned during training.=

The grokking phenomenon is an important case study for AI safety because it demonstrates that neural networks can internally reorganize themselves long after apparent convergence. A model may appear to behave one way externally while internally developing a completely different computational strategy.

In this project, Fourier analysis revealed that the transformer eventually learned structured periodic representations of modular arithmetic. This conclusion could not be obtained from test accuracy alone; it required studying the internal structure of the network directly. This demonstrates one of the main challenges in AI alignment: observing a model's external behavior is not always sufficient to understand how it truly works internally. A neural network may appear aligned while still developing unexpected internal reasoning strategies.

Mechanistic interpretability is therefore important for AI alignment because it provides tools for verifying not only what a model does, but also how it achieves its behavior internally. By understanding the learned circuits inside neural networks, researchers can better identify potentially unsafe reasoning patterns, hidden objectives, or unintended behaviors.

As artificial intelligence systems continue to scale, mechanistic interpretability may become one of the most important tools for ensuring that advanced AI systems remain transparent, understandable, and aligned with human goals.

6. Modular multiplication

Finally, the modular arithmetic task was extended from modular addition to modular multiplication in order to investigate whether the transformer could learn more complex mathematical algorithms. To allow a direct comparison with the original grokking experiment from Part 1, the same training conditions were maintained:

- 40,000 epochs
- ($p = 113$)
- Training fraction: 0.3

It is important to note that the original training notebook was initially designed for modular addition. For the purpose of this section, the code was modified using Antigravity software so that the transformer could be trained on modular multiplication instead.

The transformer was trained to compute:

$$a \times b \bmod p$$

instead of:

$$a + b \bmod p$$

For a prime modulus (p), the nonzero numbers modulo (p) form a cyclic multiplicative group. This means that every number can be written as a power of a primitive root:

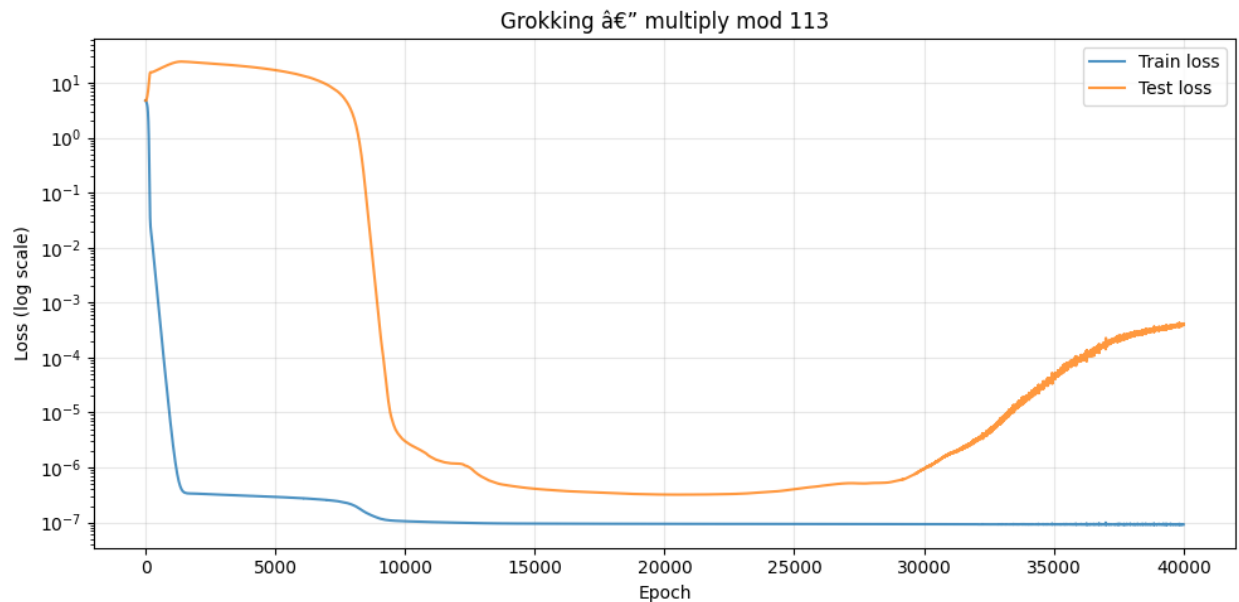
- ($a = g^i$)
- ($b = g^j$)

Then modular multiplication becomes:

$$g^i \times g^j = g^{(i+j)}$$

This suggests that the transformer may solve modular multiplication by internally mapping numbers into a cyclic or logarithmic representation, performing addition in exponent space, and then mapping back to the modular output.

The results suggest that even small transformers can discover surprisingly structured mathematical algorithms through optimization alone



Graph: Training and test loss curves for modular multiplication

The results of the modular multiplication model were surprising. The model grokked around 6,000–10,000 epochs, which is significantly earlier than the modular addition model from Part 1. This result was unexpected because modular multiplication is generally considered a more complex task than modular addition, and it was therefore expected to require more training before generalization occurred.

Additionally, near the end of training, the test loss began increasing again while the training loss remained extremely low. This behavior may indicate late-stage overfitting or instability in the learned representation after the model had already discovered a generalizable algorithmic structure.

Conclusion

This project successfully reproduced and analyzed the grokking phenomenon in small transformer models trained on modular arithmetic tasks. The experiments showed that transformers initially memorize the training data before later discovering cleaner and more generalizable algorithmic representations.

Using mechanistic interpretability techniques, the internal structure of the network was studied through embedding matrices, attention heads, MLP neuron activations, and Fourier analysis. The results revealed sparse Fourier structures and highly organized periodic patterns inside the model, demonstrating that the transformer learned compact mathematical representations rather than relying purely on memorization.

The project also investigated the three phases of grokking, memorization, circuit formation, and cleanup by tracking excluded loss, restricted loss, weight norms, and test loss throughout training. Additional experiments on training data fraction showed that delayed grokking occurs primarily in a specific regime of data scarcity.

Finally, the modular multiplication experiment extended the investigation to a more complex arithmetic task. Surprisingly, the multiplication model grokked earlier than the modular addition model, despite multiplication generally being considered more complex. However, the test loss began increasing again near the end of training, suggesting possible late-stage overfitting or instability in the learned representation. This highlights that neural networks may develop unexpected behaviors even after successful generalization.

Overall, the objectives of this project were successfully achieved. The results demonstrate that neural networks are capable of learning structured algorithmic representations through optimization alone, while also showing the importance of mechanistic interpretability for understanding how these internal computations emerge and evolve during training.

References

- [1] LessWrong. (2026, February 14). *Model multitasking: Can a model learn two different tasks...?* <https://www.lesswrong.com/posts/8qnNhepRjL9BiHHRx/model-multitasking-can-a-model-learn-two-different-tasks>
- [2] Gomezjurado Gonzalez, L. (2026). *The long delay to arithmetic generalization: When learned representations outrun behavior.* arXiv. <https://arxiv.org/html/2604.13082>